Examples and best practices

Optimizing data models for AI

Copy MCP Server

Learn how to model data to support natural language queries and enable Atlas AI agents to be reused across domains and workflows.

This article outlines how to model data to support natural language queries, accurately ground answers in authoritative sources, and enable **Atlas AI agents** to be reused across domains and workflows.

AI agents use generative AI, specifically large language models (LLMs), to solve industrial business problems, like providing insights into historical and planned maintenance and analysis of time series data for root cause analysis. The agents include prompts with instructions, industry-relevant tools, and access to industrial data stored in your CDF knowledge graph. The agents have context about the underlying data model in CDF, including the **documentation you add to the data model definition**.

Data model design

When designing a data model for AI agents, consider the following:

- **User-centric design**: Focus on the needs and expectations of the end-users who will interact with the AI agents. Understand their workflows and the types of queries they are likely to make. Would they, provided with the same context as the LLM, be able to answer the questions?
- **Clarity and simplicity**: Strive for a clear and simple data model that is easy to understand and use. Avoid ambiguity.
- **Consistency**: Maintain consistency in naming conventions, data types, and structures throughout the data model. This helps to reduce confusion and makes the model easier

Data model concepts are the building blocks of the data model and define the structure and relationships of the data. For example, Atlas AI agents are often used with these core concepts and properties:

- Assets — model the physical structure of the plant or facility, for example as an **asset hierarchy**. The assets serve as contextual anchors for other concepts like time series, files, and activities.
- Time series — link time series to assets and use clear and consistent **units**. Time series lets you trend operational behavior and ground questions about status.
- Files — link documents to assets or equipment. Files lets you ask contextual questions or summarize information.
- Activities — make sure activities like events and maintenance orders are properly typed and scoped. Activities allow agents to explain or correlate causes.
- Relations — define consistent relationships between concepts, like asset-to-file and activity-to-equipment. The relationships enable multi-hop queries and semantic linking.

Example query

To see how this works in practice, consider a prompt like: "*Is pump P-101 vibrating abnormally?*". To find and summarize the recommended intervention, an AI agent must:

- Query the knowledge graph to identify P-101.
- Search for files linked to P-101 (manuals, data sheets etc.).
- Determine vibration abnormality thresholds by searching the files.
- Query the linked time series for vibration data.
- Analyze if the time series has been outside the thresholds.

Documenting the data model

Use human-readable names

Ideally, use human-readable names for **types and properties** and avoid abbreviations. Clear and concise property names enhance the understanding of the data model.

Include short descriptions for types

Add a brief description above each **type** to give context. For instance, if the type `Event` is used for work orders and alerts, specify this in the type's docstring. This helps the LLM understand that queries about work orders and alerts refer to `Event`.

```
"""
An abstraction of an event that occurs at some point in time.
An Event can be a work order or an alert.
For example: Open Valve A, Place scaffolding
"""
type Event {..}
```

Document every property

Document every property within a . Include a clear description and provide an example value. Follow this convention:

```
  ID from the source system
  Example: 21003104
  """
  id: String
  ...
}
```

Specify enum values

If a can only have specific values (is an `Enum`), create an enum type for the property. For example:

```
enum Status {
  Open
  Closed
  Released
}

type WorkOrder {
  """
  Active status of the work order.
  """
  status: Status
}
```

Last modified on January 27, 2026

Debugging query performance
Previous

Time series in data modeling
Next